

# Projektbeschreibung

## Einleitung

Zählerdaten, welche durch Stromzähler im Schweizer Stromnetz erfasst werden, werden im standardisierten Austauschformat «sdat» zwischen Partnern (Energieberatungen, Haustechnikunternehmen, Gemeinden, Elektrizitätswerken etc.) geteilt. Die aus dem sdat-File gelesenen Werte sind immer nur Verbrauchswerte. Also relative Werte ohne Bezug zum Zählerstand. Damit ein Bezug zum Nullpunkt und damit zum effektiven Zählerstand erstellt werden kann, werden ESL-Files benötigt. In diesen Files sind die absoluten Zählerstände gespeichert. Diese beiden File-Typen sind in Textform aber sehr schwer zu interpretieren. Es soll deshalb eine Software entwickelt werden, die diese sdat-Files und ESL-Files einlesen und visualisieren kann. Ausserdem soll der Inhalt der Files als \*.csv File exportiert werden können. Wünschenswert wäre zudem, wenn die Daten im JSON-Format via POST (HTTP) auf einen Server geladen werden könnten. Oder wenn die Daten im JSON-Format als Text gespeichert werden können.

## Softwareumfang

Entwerfen und Programmieren Sie eine Software welche:

Muss:

- sdat-Files einlesen kann
- ESL-Files einlesen kann
- Die Verbrauchswerte in Zählerstände umrechnen kann
- Die Daten in einem Verbrauchsdiagramm visualisieren kann
- Die Daten in einem Zählerstandsdiagramm visualisieren kann
- Die Daten als csv exportieren kann

Nice to have:

- Die Daten via http POST auf einen Server laden kann
- Die Daten als JSON-File abspeichern kann

Das Klassendiagramm soll auch die Klassen für die Aufgaben aus «Nice to have» umfassen.

## Tipps vom Auftraggeber

Sowohl in den sdat- wie auch in den ESL-Files gibt es doppelte Datenwerte. Ich empfehle als Primärschlüssel für die Identifikation des Zählerstandes oder des Verbrauchswertes den Zeitstempel. Erstellen Sie sich eine Liste der Verbrauchswerte (sdat-Files) ohne Duplikate, welche Sie dann nach Timestamp sortieren können. Diese Liste verknüpfen Sie dann mit den absoluten Zählerständen aus den ESL-Files.

Sie haben die Daten von einem Stromzähler in einem Gebäude mit Solaranlage. Deshalb gibt es sowohl Werte für die Einspeisung (Strom von der Solaranlage ins Netz) und Werte für den Bezug (Strom von Stromnetz ins Gebäude). Für die Einspeisung werden die sdat-Files mit der **ID735** verwendet und für den Bezug vom Netz die sdat-Files mit der **ID742**. Die Zählerstände dazu finden Sie in den ESL-Files. Welche Zahlenwerte genau zu den zwei ID's gehören finden Sie in der Tabelle im Anhang.

## Anhang

### Eckpunkte sdat-File

Das sdat-File ist ein XML-File. Diverse Informationen können daraus gelesen werden. Die wichtigste Information sind die Verbrauchsdaten des im Tag *DocumentID* identifizierten Zählers. Einige wichtige Tags sind hier beschrieben:

```
<rsm:DocumentID>eslevu156407_BR2294_ID735</rsm:DocumentID>
```

Die *DocumentID* ist die eindeutige Identifizierung des Zählers.

```
<rsm:Interval>
  <rsm:StartDateTime>2019-09-24T22:00:00Z</rsm:StartDateTime>
  <rsm:EndDateTime>2019-09-25T22:00:00Z</rsm:EndDateTime>
</rsm:Interval>
```

Innerhalb der *Interval*-Tags finden Sie Start und Endzeit in UTC der Zählerwerte.

```
<rsm:Resolution>
  <rsm:Resolution>15</rsm:Resolution>
  <rsm:Unit>MIN</rsm:Unit>
</rsm:Resolution>
```

Im *Resolution*-Tag steht die Auflösung (Zeitlicher Abstand der einzelnen Messwerte)

```
<rsm:Observation>
  <rsm:Position>
```

```
    <rsm:Sequence>1</rsm:Sequence>
```

Gibt die Nummer des Messwertes an.

```
  </rsm:Position>
```

```
    <rsm:Volume>2.250</rsm:Volume>
```

Gibt die gemessene Menge/Volumen innerhalb der letzten *Resolution* wieder

```
    <rsm:Condition>21</rsm:Condition>
```

```
</rsm:Observation>
```

Am Ende des Files befinden sich mehrere *Observation*-Tags. Diese geben jeweils den Messwert im jeweiligen Zeitintervall an.

## Eckpunkte ESL-File

```
<TimePeriod end="2019-01-01T00:00:00">
```

Der Tag *TimePeriod* gibt an, von wann die Messwerte stammen.

```
<ValueRow obis="1-1:1.8.1" value="4755.3000" status="v"/>
<ValueRow obis="1-1:1.8.2" value="14460.9000" status="v"/>
<ValueRow obis="1-1:2.8.1" value="8258.1000" status="v"/>
<ValueRow obis="1-1:2.8.2" value="3543.1000" status="v"/>
<ValueRow obis="1-1:5.8.1" value="293.0000" status="v"/>
<ValueRow obis="1-1:5.8.2" value="580.0000" status="v"/>
<ValueRow obis="1-1:6.8.1" value="33.0000" status="v"/>
<ValueRow obis="1-1:6.8.2" value="8.0000" status="v"/>
<ValueRow obis="1-1:7.8.1" value="406.0000" status="v"/>
<ValueRow obis="1-1:7.8.2" value="163.0000" status="v"/>
<ValueRow obis="1-1:8.8.1" value="500.0000" status="v"/>
<ValueRow obis="1-1:8.8.2" value="1685.0000" status="v"/>
```

Pro Zähler, welcher über den *obis*-code identifiziert wird, gibt es einen *value* welcher für den Zählerstand zum Zeitpunkt in der *TimePeriod* steht.

```
</TimePeriod>
```

## Zuordnung DocumentID und obis-code

| Obis                                                                                   | DocumentID   |
|----------------------------------------------------------------------------------------|--------------|
| <b>1-1:1.8.1 (Bezug Hochtarif)</b><br><b>1-1:1.8.2 (Bezug Niedertarif)</b>             | <b>ID742</b> |
| <b>1-1:2.8.1 (Einspeisung Hochtarif)</b><br><b>1-1:2.8.2 (Einspeisung Niedertarif)</b> | <b>ID735</b> |

Die Zählerstände von jeweils zwei Obis-Codes (Hochtarif und Niedertarif) müssen addiert werden, um den effektiven Zählerstand für Einspeisung resp. des Bezugs zu erhalten. Diese Zählerstände aus dem ESL-File entsprechen dann den aufsummierten Verbrauchswerten aus den sdat-Files.

## Dokumentation JSON-Format

```
[
{
  "sensorId": "ID742",
  "data": [
    {
      "ts": "1503495302",
      "value": 82.03
    },
    "..."
  ]
},
{
  "..."
}
]

[
{
  "sensorId": "ID735",
  "data": [
    {
      "ts": "1503495303",
      "value": 1129336.0
    },
    "..."
  ]
}
]
```

Pro Sensorid werden im kleinstmöglich verfügbaren Zeitabstand die Zählerstände dokumentiert. Der Zählerstand jeweils als Absolutwert und der Timestamp in UTC.

## Dokumentation CSV-Format

Filename entspricht der SensorID. Pro File werden im kleinstmöglich verfügbaren Zeitabstand die Zählerstände dokumentiert. Der Zählerstand jeweils als Absolutwert und der Timestamp in UTC.

```
timestamp, value
1503495303, 1129336.0
1503496303, 1129339.0
1503497303, 1129340.0
...
```

## Generelle Tipps von Roger Bünzli

Roger Bünzli ist Hobby-Java-Entwickler er hat sich bereits einiges über die geforderte Applikation überlegt. Er denkt, es macht Sinn, dass es eine Klasse «Messwert» gibt, in welchem der Timestamp, der Absolute wie auch der Relative Messwert gespeichert wird. Dieser Messwert wird dann pro Stromzähler in einer geeigneten Datenstruktur abgelegt. Herr Bünzli hat dabei an ein `TreeMap<Key,Value>` gedacht, da ein `TreeMap` keine Duplikate enthalten kann. Wäre jetzt der Timestamp gleichzeitig auch Key, wäre sichergestellt, dass keine doppelten Werte vorkommen können.

Roger Bünzli hat auch schon mal mit den XML-Files und Java gespielt, folgender Lerntagebuch Eintrag hat er Ihnen noch zur Verfügung gestellt:

### Kalenderwoche: Woche vom 6.9 bis zum 12.9

**Tätigkeit:** Ich habe mir vorgenommen Daten aus einem XML File auszulesen und diese dann in Java abzuspeichern. Dazu habe ich einen Ordner in meinem Workspace wo alle Files drin liegen. Dann mache ich eines nach dem anderen auf und hole Informationen heraus. Aus den Files möchte ich Messwerte holen, die ich danach miteinander verrechnen muss. Es gibt 2 unterschiedliche XML Files und beide müssen unterschiedlich ausgelesen werden.

### Was konnte ich lernen:

Wie ich ein XML File in Java einlese und dieses dann nach den XML-Nodes durchsuchen kann. Dazu verwende ich die `DocumentBuilder` Klasse und speichere die geparsen XML dann in ein Document.

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();

// Load the input XML document, parse it and return an instance of the
// Document class.
Document document = builder.parse(file);
document.getDocumentElement().normalize();

// Get Time
NodeList timePeriodList = document.getElementsByTagName("TimePeriod");
String endDateTime = timePeriodList.item(index: 0).getAttributes().item(index: 0).getTextContent();
time.setTime(df.parse(endDateTime));
```

Habe ich ein Array von Nodes, kann ich mittels for-Schleife über die Liste gehen und dann mit `getElementsByTagName` auslesen.

```
//Get all Entries
NodeList observationList = document.getElementsByTagName("rsm:Observation");

for (int temp = 0; temp < observationList.getLength(); temp++) {
    Node observation = observationList.item(temp);
    if (observation.getNodeType() == Node.ELEMENT_NODE) {
        Element observationElement = (Element) observation;
        int sequence = Integer.parseInt(observationElement.getElementsByTagName("rsm:Sequence").item(0).getTextContent());
        double value = Double.parseDouble(observationElement.getElementsByTagName("rsm:Volume").item(0).getTextContent());

        dataHandler.addData(calStart.getTime().getTime(), new MeteringData(value, calStart.getTime().getTime(), id));
        calStart.add(Calendar.MINUTE, amount: 15);
    }
}
```

**Welches Wissen fehlt mir noch:** Ich muss noch herausfinden, ob es auch die Möglichkeit gibt, das XML Automatisch in Objekte zu parsen. Jetzt mache ich das Ganze noch manuell in dem ich nur die Werte hole die ich brauche...

**Welches Wissen möchte ich festhalten:**

Wenn ich folgendes XML habe:

```
<?xml version="1.0" encoding="UTF-8"?>
<ESLBillingData>
<Header version="1.0" created="2019-01-31T00:00:00" swSystemNameFrom="ESL-
EVU" swSystemNameTo="ESL-EVU" />
<Meter factoryNo="38157930" internalNo="38157930">
<TimePeriod end="2019-01-01T00:00:00">
<ValueRow obis="1-1:1.6.1" valueTimeStamp="2018-12-
05T08:45:00" value="6.3000" status="V"/>
<ValueRow obis="1-1:1.6.2" valueTimeStamp="2018-12-
08T10:45:00" value="5.3000" status="V"/>
<ValueRow obis="1-1:1.8.1" value="4755.3000" status="V"/>
<ValueRow obis="1-1:1.8.2" value="14460.9000" status="V"/>
<ValueRow obis="1-1:2.6.1" valueTimeStamp="2018-12-
22T12:00:00" value="1.4000" status="V"/>
<ValueRow obis="1-1:2.6.2" valueTimeStamp="2018-12-
22T12:00:00" value="1.4000" status="V"/>
<ValueRow obis="1-1:2.8.1" value="8258.1000" status="V"/>
<ValueRow obis="1-1:2.8.2" value="3543.1000" status="V"/>
<ValueRow obis="1-1:5.8.1" value="293.0000" status="V"/>
<ValueRow obis="1-1:5.8.2" value="580.0000" status="V"/>
<ValueRow obis="1-1:6.8.1" value="33.0000" status="V"/>
<ValueRow obis="1-1:6.8.2" value="8.0000" status="V"/>
<ValueRow obis="1-1:7.8.1" value="406.0000" status="V"/>
<ValueRow obis="1-1:7.8.2" value="163.0000" status="V"/>
<ValueRow obis="1-1:8.8.1" value="500.0000" status="V"/>
<ValueRow obis="1-1:8.8.2" value="1685.0000" status="V"/>
</TimePeriod>
</Meter>
</ESLBillingData>
```

Wenn ich von hier den Wert TimePeriod end=xxxx Wert will mache ich das so:

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();

// Load the input XML document, parse it and return an instance of the
// Document class.
Document document = builder.parse(file);
document.getDocumentElement().normalize();

// Get Time
NodeList timePeriodList = document.getElementsByTagName("TimePeriod");
String endDateTime = timePeriodList.item(index: 0).getAttributes().item(index: 0).getTextContent();
time.setTime(df.parse(endDateTime));
```